

Guia de Implementação — Shark Hub



Guia de Implementação — Shark Hub

Manual para **você terminar o trabalho pensando e programando** — não copiando. Aqui não tem código Java pronto: tem objetivo, perguntas para te guiar, lógica em pseudocódigo e dicas de armadilhas. O código você escreve. 💪

Como usar este guia

- Cada tarefa tem 5 partes:
 1. 🎯 **Objetivo** — o que precisa acontecer.
 2. ? **Pense primeiro** — perguntas para você responder antes de codar.
 3. 🧩 **Lógica (pseudocódigo)** — o esqueleto do raciocínio, em português.
 4. ⚠️ **Armadilhas** — erros comuns para evitar.
 5. ✅ **Como testar** — como saber que funcionou.
- Marque os [] conforme terminar. Ver a lista encher motiva — é de propósito.
- **Regra de ouro:** se você só copiou e não consegue explicar em voz alta, refaça. Lembre que a apresentação vale **4,0** e cobra que você *domine* o código.

Convenção do pseudocódigo

Não é Java. É só raciocínio. Você traduz para Java.

```
SE (condição) ENTÃO ... SENÃO ...  
PARA CADA item DE inicio ATE fim FAÇA ...  
ENQUANTO (condição) FAÇA ...  
RETORNE valor  
FUNÇÃO nome(parâmetros) -> tipo do retorno
```



Mapa de progresso

Faça **em ordem**. Cada fase usa o que a anterior construiu.

- ☐ **Fase 1 — Consertar o que já existe** (aquecimento, ganha confiança)
- ☐ **Fase 2 — Completar as classes** (a "base de dados" do sistema)

- ☐ **Fase 3 — Funcionalidades que faltam** (o coração: opções 4, 5, 7, 8, 9)
- ☐ **Fase 4 — Funcionalidade Inovadora: IPU** (opção 10, vale 1,0)
- ☐ **Fase 5 — Documentação** (README + Uso de IA, valem 2,0 juntos)
- ☐ **Fase 6 — Desafio extra** (opcional, diferencial na apresentação)

💡 **Truque mental:** sempre que travar, procure no seu próprio código uma função parecida que **já funciona** e use de molde. Quase tudo aqui é variação de `listarInvestidores` e `buscarInvestidor`.

Fase 0 — Preparação (5 min)

Antes de mexer, garanta que você sabe rodar e que está tudo compilando:

```
javac *.java      # compila tudo
java Main         # roda
```

- ☐ Compilou sem erro e o menu apareceu.
- ☐ Você consegue cadastrar 1 investidor e 1 startup pelo menu.

Sempre **compile e rode depois de cada tarefa pequena**. Não acumule mudanças sem testar — é assim que aparecem aqueles bugs impossíveis de achar.

Fase 1 — Consertar o que já existe

São bugs pequenos, mas que tiram pontos em "Execução sem erros". Comece por aqui: é rápido e te reconecta com o código.

1.1 — O bug do `Scanner` (título do pitch vem vazio)

Arquivo: `Main.java`, dentro de `adicionarPitch` (~linha 193).

🎯 Fazer o título e as notas serem lidos corretamente.

? Pense primeiro:

- Quando você usa `nextInt()` / `nextDouble()`, o "Enter" que você apertou **fica sobrando** no buffer. O que o próximo `nextLine()` vai ler então?

- Abra `cadaststrarInvestidor` (linhas 86–91). O que ele faz logo depois de `nextInt()` que o `adicionarPitch` **esqueceu** de fazer?

🧩 Lógica:

```
leia número
LIMPE o buffer (consuma o Enter que sobrou)
leia o texto
```

✅ Teste: cadastre um pitch e confira que o título **não** ficou em branco.

1.2 — `verificarEmail` não detecta e-mail repetido de verdade

Arquivo: `Main.java`, `verificarEmail` (linhas 176–187).

🔗 Bloquear cadastro com e-mail já existente (o PDF exige mensagem de erro).

? Pense primeiro:

- Para afirmar que um e-mail é **único**, com quantos investidores você precisa comparar? Só com o primeiro, ou com **todos**?
- O `return` atual está **dentro** do laço. Isso faz a decisão ser tomada cedo demais. Por quê?

🧩 Lógica sugerida (uma função que responde "esse e-mail já existe?"):

```
FUNÇÃO emailJaExiste(email) -> verdadeiro/falso
  PARA CADA investidor cadastrado FAÇA
    SE o email dele for igual ao informado ENTÃO RETORNE verdadeiro
  RETORNE falso // só chega aqui depois de checar TODOS
```

E no cadastro:

```
ENQUANTO emailJaExiste(email) FAÇA
  avise "e-mail já cadastrado" e peça outro
```

⚠️ Armadilha: comparar String com `==` não funciona em Java. Use `.equals(...)` (você já faz isso em `buscarInvestidor`).

✅ Teste: cadastre dois investidores com o mesmo e-mail. O segundo deve ser barrado.

1.3 — `toString()` mostra lixo tipo `Investidor@1b6d3586`

Arquivos: `Investidor.java` (33–36) e `Startup.java` (58–61).

🔗 Quando imprimir um investidor/startup, mostrar dados legíveis.

? Pense: o `toString` atual chama `super.toString()`, que devolve o endereço de memória. O que você **gostaria** que aparecesse no lugar? (ex.: nome + e-mail)

🧩 Lógica: monte e retorne uma String com os atributos que fazem sentido.

✅ Teste: na opção 5 (Consultar Startup), o investidor deve aparecer pelo **nome**, não pelo código.

1.4 — `adicionarPitch` **escolhe a startup errada**

Arquivo: `Main.java`, `adicionarPitch` (linhas 204–211).

🎯 Associar o pitch à startup **certa**, escolhida pelo ID.

? Pense primeiro:

- Hoje o código faz `startups[escolher]`, usando o **ID digitado como índice** do vetor. Se a startup de ID 7 estiver na posição 0, o que acontece? E se não existir startup nenhuma com aquele ID?
- Você já tem `buscarInvestidor(nome)`. Que função análoga falta criar para startups?
- O laço atual imprime "registrado/tente novamente" para **cada** startup. Faz sentido? O que ele deveria fazer: encontrar **uma** ou reagir a **todas**?

🧩 Lógica:

```
crie FUNÇÃO buscarStartupPorId(id) -> Startup (ou null se não achar)
no adicionarPitch:
    SE não há startups cadastradas ENTÃO avise e volte
    peça o id da startup
    startup = buscarStartupPorId(id)
    SE startup == null ENTÃO avise "não encontrada" e volte
    crie o Pitch com essa startup, guarde no vetor, incremente o total
```

⚠️ Apague aquela mensagem trocada ("Startup cadastrada com sucesso" dentro do pitch).

✅ Teste: cadastre 2 startups, adicione um pitch à de ID maior e confira (com `listarPitches`) que vinculou na certa.

☐ **Fase 1 concluída** — programa roda sem esses bugs.

Fase 2 — Completar as classes 🧱

As funcionalidades só funcionam se as classes existirem direito. Construa a base aqui.

2.1 — Classe `Investimento` (está com atributos errados)

Arquivo: `Investimento.java`.

🎯 Modelar um investimento conforme o PDF: `id` (int), `valor` (double), `percentualParticipacao` (double), `startup` (Startup).

? Pense: use a classe `Pitch.java` como **molde** — ela já tem a estrutura ideal (atributos → construtor → getters). O que muda são os campos.

🧩 Estrutura:

```
atributos: id, valor, percentualParticipacao, startup
construtor que recebe e preenche os 4
getters (e setters – o PDF pede "métodos para manipulação")
```

🧠 Sobre o método "calcular valor total investido na startup": isso significa **somar todos os investimentos daquela startup**. Decisão de projeto sua: essa soma pode morar aqui (recebendo o vetor de investimentos + a startup) **ou** na classe `Valuation` (2.2). Escolha um lugar só e seja coerente. Anote sua escolha para explicar na apresentação.

2.2 – Classe `Valuation` (está vazia)

Arquivo: `Valuation.java`.

🎯 Calcular o total captado por uma startup (soma dos investimentos recebidos).

🧩 Lógica do método de cálculo:

```
FUNÇÃO totalCaptado(vetor de investimentos, quantidade, startupAlvo) -> double
    soma = 0
    PARA CADA investimento existente FAÇA
        SE o investimento for da startupAlvo ENTÃO soma = soma + valor dele
    RETORNE soma
```

? Pense: como decidir se um investimento "é da startupAlvo"? Comparar por **ID** da startup é mais seguro do que comparar objetos. Por quê você acha que é mais seguro?

2.3 – Criar o vetor de investimentos no `Main`

Arquivo: `Main.java` (perto das linhas 6–13).

🎯 Ter onde guardar os investimentos (o PDF exige **vetor de objetos**).

🧩 Espelhe o que já existe para pitches:

```
um vetor de Investimento com tamanho máximo
um contador totalInvestimentos = 0
```

2.4 — Setters do `Pitch`

Arquivo: `Pitch.java`. Adicione os setters dos 5 atributos (o PDF pede "métodos para manipulação dos atributos"). Use os getters como molde.

2.5 — `Startup.calcularExistencia` (hoje não calcula nada)

Arquivo: `Startup.java` (52–56).

🎯 Retornar há quantos anos a startup existe, a partir da `data` ("dd/mm/aaaa").

? Pense:

- A data está guardada como **texto**. Como extrair só o **ano** de `"15/03/2020"`? (pesquise `split` ou `substring`)
- Versão simples: `ano de hoje - ano de fundação`. Versão caprichada: considerar mês/dia. Comece pela simples.

🧩 Lógica (versão simples):

```
extraia o ano da String da data
RETORNE anoAtual - anoFundacao
```

⚠️ O método hoje é `static` e pede a data por teclado — repense: faz mais sentido ele usar o **próprio atributo** `data` da startup. Esse método deveria precisar de `Scanner`?

☐ **Fase 2 concluída** — todas as 5 classes completas e o `Main` com os 4 vetores (investidores, startups, pitches, investimentos).

Fase 3 — Funcionalidades que faltam ❤️

Agora dá pra ligar as opções do menu. Para cada uma: descomente a chamada no `switch` (linhas 51–71 do `Main`) e crie o método.

3.1 — Opção 4: Adicionar Investimento

🎯 Registrar um investimento numa startup e guardá-lo no vetor.

? Pense: qual função da Fase 1 você vai **reusar** para escolher a startup pelo ID? (dica: `buscarStartupPorId`).

🧩 Lógica:

```
SE não há startups ENTÃO avise e volte  
leia id, valor, percentualParticipacao (cuidado com o bug do Scanner!)  
valide: valor > 0 ? percentual entre 0 e 100 ?  
escolha a startup pelo ID (reuse a busca); SE não achar, avise e volte  
crie o Investimento, guarde no vetor, incremente o total  
confirme pro usuário
```

✅ Teste: adicione 2 investimentos na mesma startup; você vai usá-los no Valuation.

3.2 — Opção 7: Consultar Investimentos

🎯 Listar todos os investimentos e seus valores.

🧩 Use `listarInvestidores` (linhas 144–157) como **molde**: checa se está vazio, percorre o vetor, imprime cada um. Mostre também a qual startup pertence (pelo nome).

3.3 — Opção 8: Calcular Valuation

🎯 Mostrar o total captado por uma startup escolhida.

🧩 Lógica:

```
escolha a startup pelo ID  
chame o método de total da classe Valuation (2.2)  
exiba o resultado
```

❓ Pense: o que mostrar se a startup não recebeu nenhum investimento ainda? (Dica: zero é uma resposta válida — não deixe quebrar.)

3.4 — Opção 5: melhorar o "Consultar Startup"

O PDF exige mostrar, para a startup: **dados + investidor responsável + pitches + investimentos**. Hoje só lista o básico.

🎯 Pedir uma startup (por ID) e exibir o "dossiê" completo dela.

🧩 Lógica:

```
escolha a startup pelo ID  
imprima os dados dela e o investidor (nome)  
PARA CADA pitch existente FAÇA  
    SE o pitch for dessa startup ENTÃO imprima o pitch (e a nota média!)  
PARA CADA investimento existente FAÇA  
    SE o investimento for dessa startup ENTÃO imprima o investimento  
mostre o total captado (reuse o Valuation)
```

? Pense: esse padrão "percorrer um vetor e filtrar pelos que pertencem à startup X" vai se repetir MUITO daqui pra frente. Entendeu bem? Ele é a base da Fase 4.

3.5 — Opção 9: Ranking de Startups

🎯 Listar as startups da "melhor" para a "pior" segundo um critério.

? Pense primeiro (decisão de projeto):

- Por qual critério ordenar? Total captado? Média dos pitches? A **pontuação do IPU** (Fase 4)? Escolha e justifique. (Dica: se você fizer o IPU primeiro, o ranking vira só "ordenar pela pontuação do IPU" — talvez valha a pena trocar a ordem.)

🧩 Como ordenar sem saber sorting avançado — algoritmo "ache o maior repetidamente":

```
marque todas as startups como "ainda não listadas"
REPITA (quantas startups houver):
    procure, entre as não listadas, a de maior pontuação
    imprima ela
    marque como listada
```

⚠ Não precisa ser o algoritmo mais rápido do mundo. Precisa estar **correto** e você **entender**.

☐ **Fase 3 concluída** — todas as opções do menu funcionam.

Fase 4 — 🌟 Funcionalidade Inovadora: IPU (opção 10)

Vale **1,0** e é o destaque da apresentação. "Índice de Potencial de Unicornização": o sistema avalia e classifica as startups sozinho.

Não tente fazer tudo de uma vez. São **5 mini-tarefas**, cada uma é um laço parecido com os que você já domina.

Passo A — Definir a fórmula da pontuação (a parte criativa)

🎯 Toda startup precisa de uma **pontuação** numérica. Você inventa a fórmula — e defende ela.

? Pense: a tabela do PDF vai até "acima de 500". Sua fórmula precisa gerar números nessa faixa. O que faz uma startup ser boa? Provavelmente: **notas altas nos pitches** e **muito investimento captado**.

🧩 Exemplo de fórmula (ajuste os pesos do seu jeito!):


```
pontuação = (média das notas dos pitches da startup) * pesoA
              + (total captado pela startup) * pesoB
```

Escolha `pesoA` e `pesoB` testando até os números caírem em faixas variadas. **Anote o raciocínio** — vão te perguntar na apresentação "por que essa fórmula?".

Passo B — Classificar cada startup

🧩 Com a pontuação em mãos:

```
SE pontuação <= 100      -> "Startup Inicial"
SENÃO SE pontuação <= 250 -> "Startup Promissora"
SENÃO SE pontuação <= 500 -> "Startup em Crescimento"
SENÃO                    -> "Potencial Unicórnio"
```

💡 Faça disso uma **função** `classificar(pontuacao) -> String`. Você vai chamar em vários lugares.

Passo C — Os indicadores que o PDF pede

Cada um é um laço de "encontrar o maior" ou "somar". Faça um por vez:

- ☐ **Startup mais promissora** → a de **maior pontuação**.
- ☐ **Startup com maior volume de investimentos** → a de **maior total captado** (some os investimentos de cada uma e compare).
- ☐ **Investidor com maior valor investido** → para cada investidor, some os investimentos das startups ligadas a ele; veja quem tem a maior soma.
- ☐ **Ranking das startups** → reuse a Fase 3.5, ordenando pela pontuação.
- ☐ **Classificação de cada startup** → percorra todas e imprima nome + pontuação + `classificar(...)`.

🧩 Molde geral de "encontrar o maior":

```
guarde "melhor" = nenhum, "melhorValor" = -1
PARA CADA startup FAÇA
    valor = (pontuação OU total captado, conforme o indicador)
    SE valor > melhorValor ENTÃO melhor = essa startup, melhorValor = valor
imprima "melhor"
```

⚠️ E se **não houver** startups/investimentos? Trate o vazio em cada indicador, senão quebra.

- ☐ **Fase 4 concluída** — opção 10 mostra os 5 indicadores e a classificação.

Fase 5 — Documentação (README 1,0 + IA 1,0)

Pontos "fáceis" que muita gente perde por esquecer. Não pule.

5.1 — Completar o `README.md`

Confira no PDF a lista exigida. Faltam:

- ☐ **Nome e número de matrícula** de **todos** os integrantes (o trabalho é dupla/trio — você está sozinho no README).
- ☐ Descrição da aplicação (já tem — revise).
- ☐ **Fontes utilizadas** (materiais de aula, sites, etc.).
- ☐ **Lições aprendidas**.
- ☐ **Dificuldades encontradas** e **como você as superou**.
- ☐ Atualizar a lista de funcionalidades (hoje várias estão como `[ ]`) e a estrutura de arquivos (faltam `Pitch`, `Investimento`, `Valuation`).

5.2 — Seção "Uso de IA" (o PDF cobra, e avisa que omitir prejudica a nota)

Crie uma seção no README (ou arquivo à parte) respondendo:

- ☐ **Ferramentas** usadas (ex.: Claude Code).
- ☐ **Objetivo** do uso (ex.: "entender o que faltava e me guiar a implementar").
- ☐ **Principais prompts** que você usou.
- ☐ **Principais respostas** obtidas.
- ☐ **Reflexão**: o que a IA ajudou, o que você aprendeu, o que faria diferente.

 Dica honesta: como você está usando este guia, descreva exatamente isso — que a IA te orientou mas **você** escreveu o código. Isso é verdade e fica forte na reflexão.

- ☐ **Fase 5 concluída**.
-

Fase 6 — Desafio extra (opcional) 🚀

Simulador de crescimento da startup por 5 anos, com percentual informado pelo usuário. Não é obrigatório, mas brilha na apresentação.

🧩 Lógica:

```
peça o valor atual (ex.: total captado) e o percentual de crescimento anual
PARA ano DE 1 ATE 5 FAÇA
    valor = valor * (1 + percentual/100)
    imprima "Ano X: valor"
```

? Pense: o que aconteceria se usasse `percentual` em vez de `percentual/100`? Teste e veja.

✅ Checklist final antes de entregar

- ☐ `javac *.java` compila **sem nenhum** erro nem warning grave.
- ☐ Rodei o menu inteiro, opção por opção (0 a 10), e nenhuma quebrou.
- ☐ Testei os casos chatos: listas vazias, IDs que não existem, e-mail repetido, nota fora da faixa.
- ☐ O programa só encerra na opção **0** (teste).
- ☐ Todos os dados estão em **vetores de objetos** (4 vetores no `Main`).
- ☐ A funcionalidade inovadora (IPU) funciona e está **integrada** (usa os dados reais).
- ☐ README completo + seção de IA.
- ☐ **Eu consigo explicar cada parte do código em voz alta** (ensaiei a apresentação de 5 min).

🧭 Ordem recomendada (resumo)

1. Fase 1 (bugs) → 2. Fase 2 (classes) → 3. Fase 3 (opções 4,7,8,5) →
2. **Fase 4 (IPU)** → 5. volte e faça o Ranking 3.5 usando a pontuação do IPU →
3. Fase 5 (docs) → 7. Fase 6 (se sobrar tempo).

Você já tem ~60% da estrutura pronta. O resto é **repetir padrões que você mesmo já escreveu**.
Uma fase por vez, compilando sempre. Você termina isso. 🐼